UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA

PROGRAMACIÓN II

MANUAL TÉCNICO

GUALI TUTOR - FASE 3

Estudiante: Dulce Rocio Sinay Mayen

Carné: 3190-24-18636

Repositorio público: https://github.com/dsinaym-prog/GualiTutor-Fase3

Tutorial público: https://github.com/dsinaym-prog/GualiTutor-Fase3/blob/main/TUTORIAL.md

1. Introducción

Guali Tutor es una aplicación móvil Android desarrollada en Java para consultar lecciones de programación, editar código Java y revisar información guardada mediante una API REST. El Front-End se implementó en Android Studio con interfaces XML y pantallas independientes. Este manual documenta la organización, el diseño, la integración y las pruebas visibles en las capturas del proyecto.

2. Objetivo

Documentar los pasos y componentes utilizados para construir el Front-End de Guali Tutor Fase 3, explicar el funcionamiento de cada módulo y mostrar evidencias reales de la aplicación en el emulador Android.

3. Requisitos y herramientas

Herramientas: Android Studio, SDK de Android, emulador Android (en las capturas se utiliza Medium Phone API 37), JDK para el desarrollo, conexión local con la API REST de Spring Boot y MariaDB como base de datos del backend. El proyecto Android utiliza Gradle y código Java/XML.

Condición para las funciones conectadas: la API debe estar iniciada en el equipo y accesible desde el emulador. En el entorno utilizado se configuró el acceso al servidor local por el puerto 8080; desde el emulador Android se emplea la dirección especial 10.0.2.2 para llegar al equipo anfitrión.

4. Estructura del proyecto

En Android Studio, el módulo app contiene el código Java en src/main/java/com/guali/tutor/android, los diseños XML en src/main/res/layout y el manifiesto de Android. Entre las clases utilizadas están MainActivity, LeccionesActivity, EditorActivity, HistorialActivity, ConfiguracionActivity y ApiClient. Cada pantalla se relaciona con su diseño XML y con los eventos de sus botones.

5. Desarrollo paso a paso del Front-End

Paso 1. Diseño de la pantalla principal

Se abrió el archivo activity_main.xml en el editor visual de Android Studio. Se organizó una interfaz vertical con un título, una breve descripción y botones de navegación. La clase MainActivity permite abrir las secciones de lecciones, historial y configuración; el botón de ayuda presenta instrucciones de uso. Los avisos amarillos visibles en el diseñador corresponden a recomendaciones de Android Studio y no demuestran por sí solos un fallo de compilación.

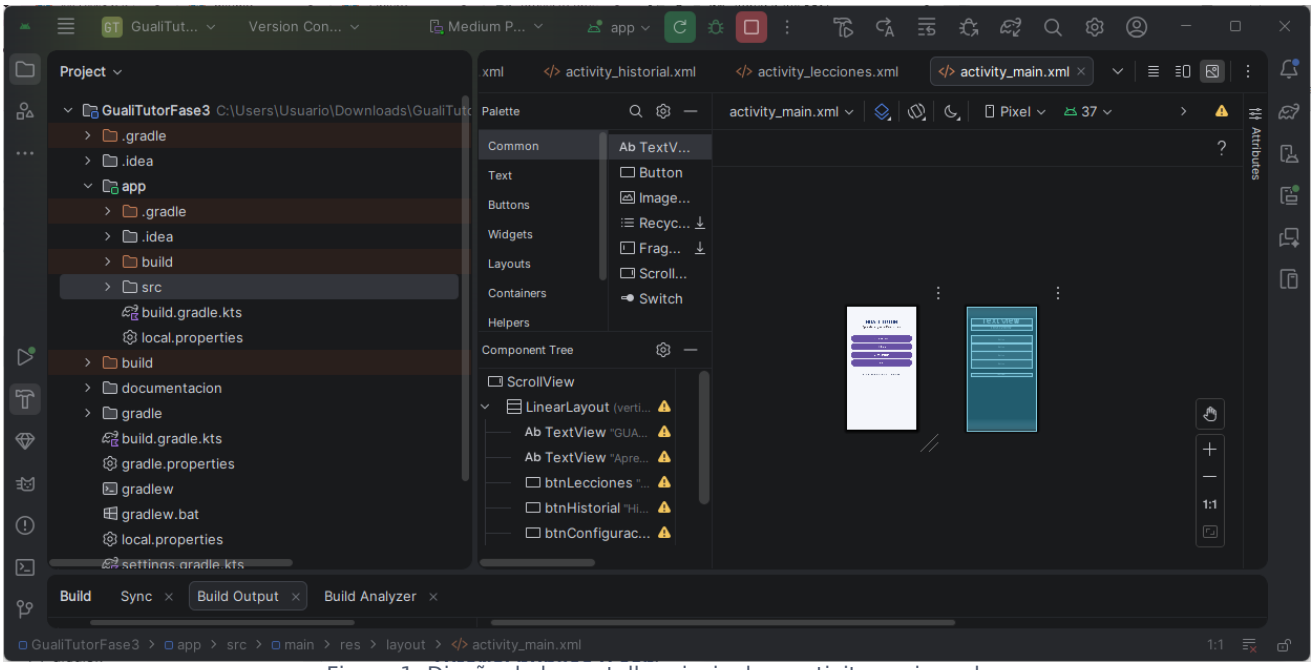


Figura 1. Diseño de la pantalla principal en activity_main.xml.

Paso 2. Diseño de la pantalla Mis lecciones

Se configuró activity_lecciones.xml con encabezado, área de listado, indicador de estado y botones para actualizar o regresar. La clase LeccionesActivity consulta el endpoint de lecciones de la API REST y presenta los resultados recibidos. Al seleccionar una lección se abre el editor, enviando sus datos para mostrar el objetivo y la ruta registrada.

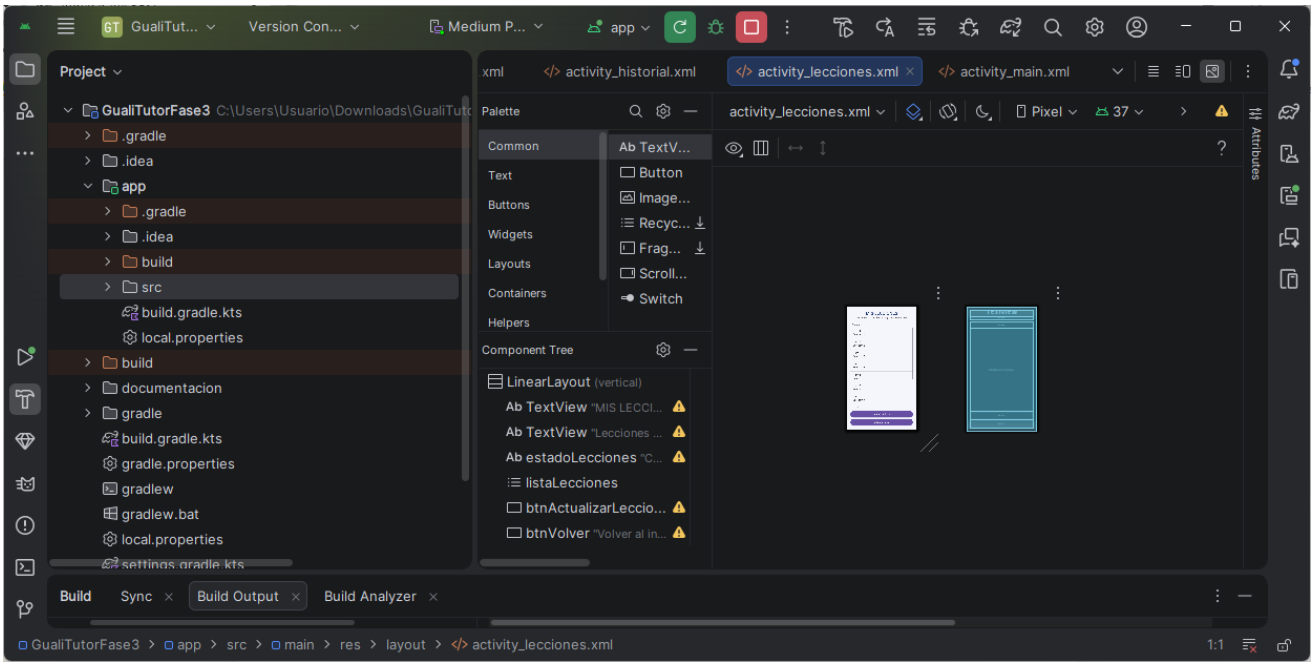


Figura 2. Diseño de la pantalla de lecciones en activity_lecciones.xml.

Paso 3. Diseño de la pantalla Historial

En activity_historial.xml se preparó el espacio para mostrar registros de actividad, con controles para actualizar y volver al inicio. HistorialActivity solicita al servidor los registros persistidos y muestra los datos devueltos, como el identificador de la lección, la fecha de apertura y el estado. La pantalla permite verificar que la información proviene de la API y no de un listado fijo en la interfaz.

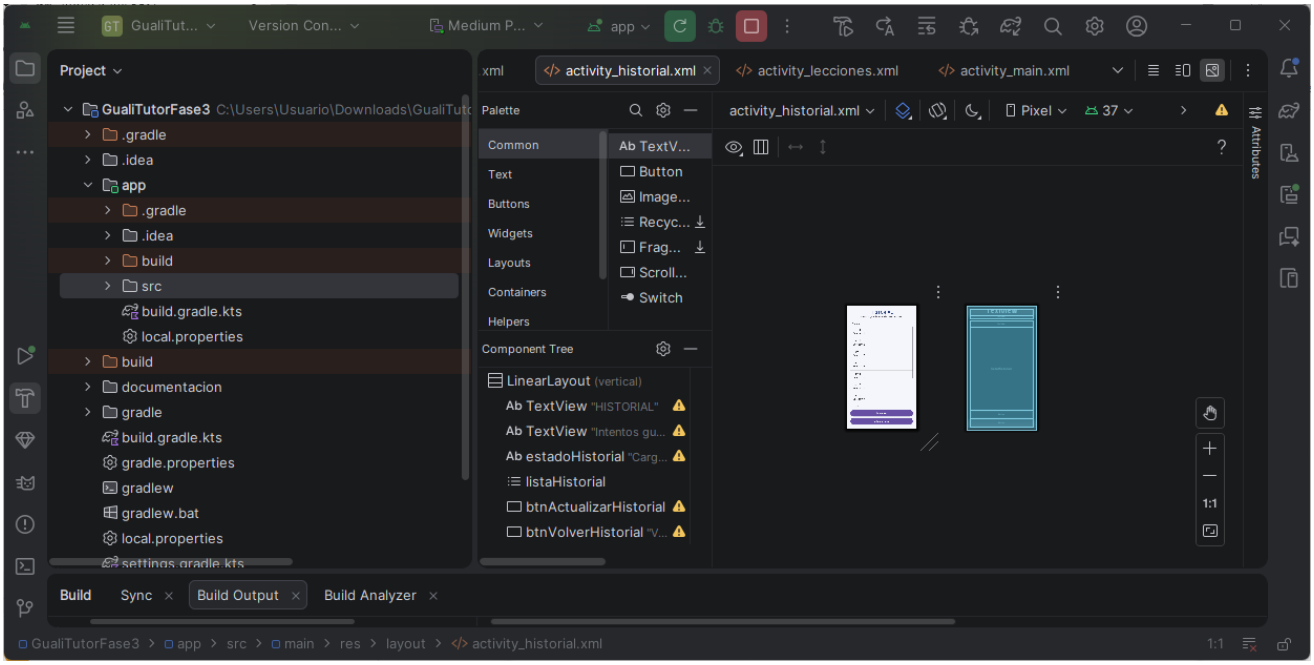


Figura 3. Diseño XML de la pantalla Historial.

Paso 4. Comprobación de Mis lecciones en el emulador

Se ejecutó la aplicación en el emulador Android y se ingresó a Mis lecciones. La captura muestra dos entradas de leccion-01 con objetivo COMPILAR_EXITOSO. Esto confirma que el listado se presenta en pantalla; la repetición de la lección corresponde a los datos recibidos y debe revisarse en la base de datos si se requiere eliminar duplicados. El botón Actualizar lista vuelve a consultar la información.

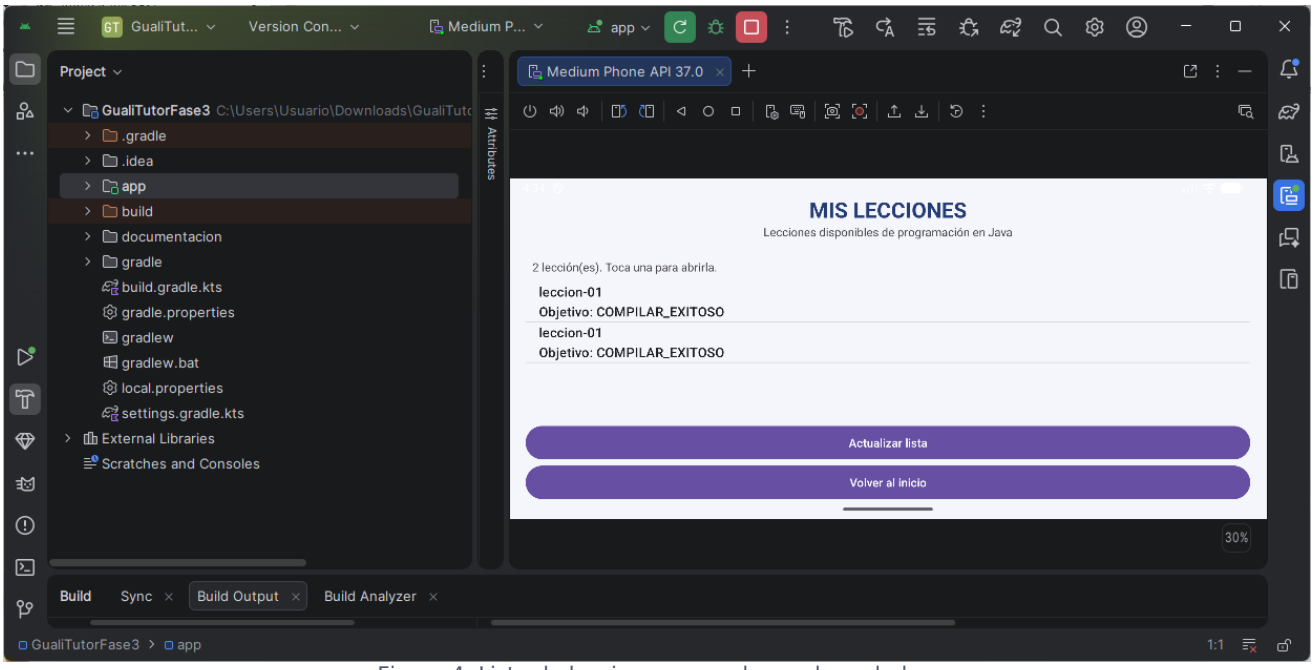


Figura 4. Lista de lecciones cargada en el emulador.

Paso 5. Editor de código y compilación

Al tocar una lección se abre EditorActivity. Esta pantalla presenta nombre, objetivo, ruta de origen, campo para escribir código Java, botones Compilar y Ejecutar, consola de mensajes y botón para cerrar la lección. La compilación se conectó mediante una petición HTTP POST al endpoint /api/compilacion/compilar; el backend recibe el texto del código, utiliza el compilador Java disponible en el JDK y devuelve el resultado o los diagnósticos. En pruebas anteriores, la consola mostró el mensaje de compilación correcta. La captura incluida aquí muestra el editor en estado inicial, con la consola lista. Ejecutar permanece deshabilitado porque todavía no existe un servicio aislado de ejecución de programas; no debe presentarse como una función implementada.

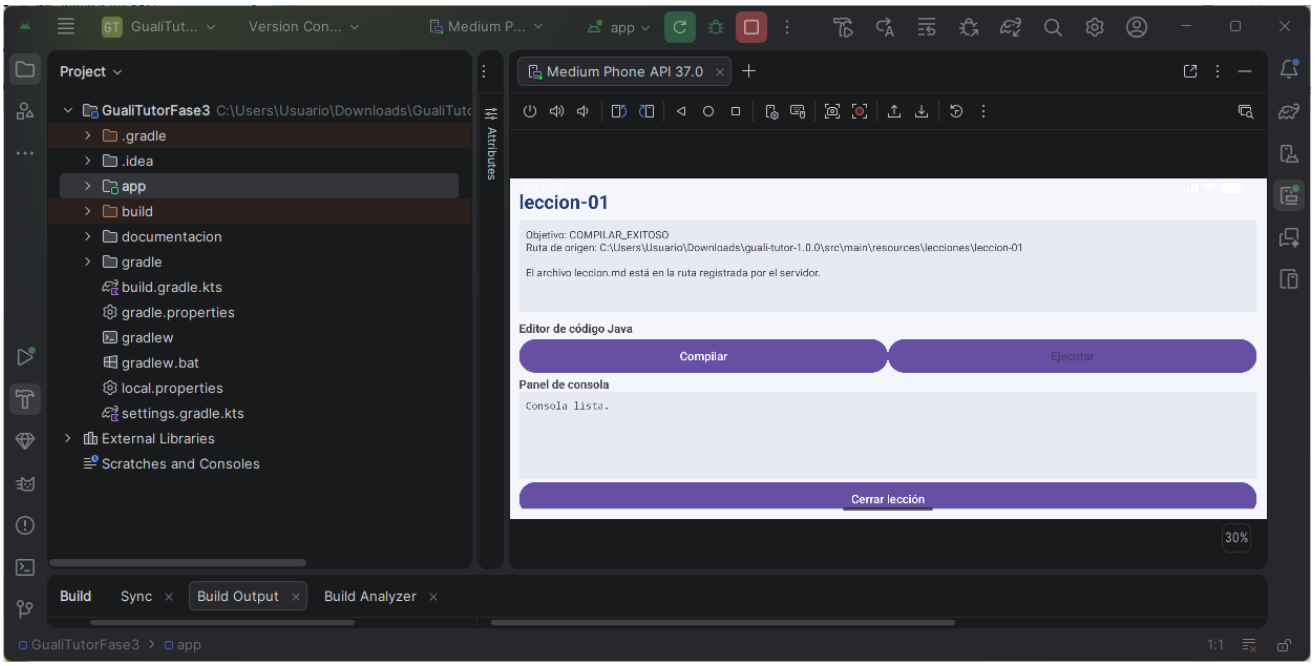


Figura 5. Pantalla de la lección con editor, controles y consola.

Paso 6. Consulta del historial guardado

Se accedió a Historial desde el menú principal. La aplicación consultó los datos del backend y mostró 7 registros persistidos, con identificador de lección, apertura y estado PENDIENTE. El botón Actualizar permite repetir la consulta; Volver al inicio retorna al menú. Esta evidencia demuestra la lectura de registros almacenados en el servidor.

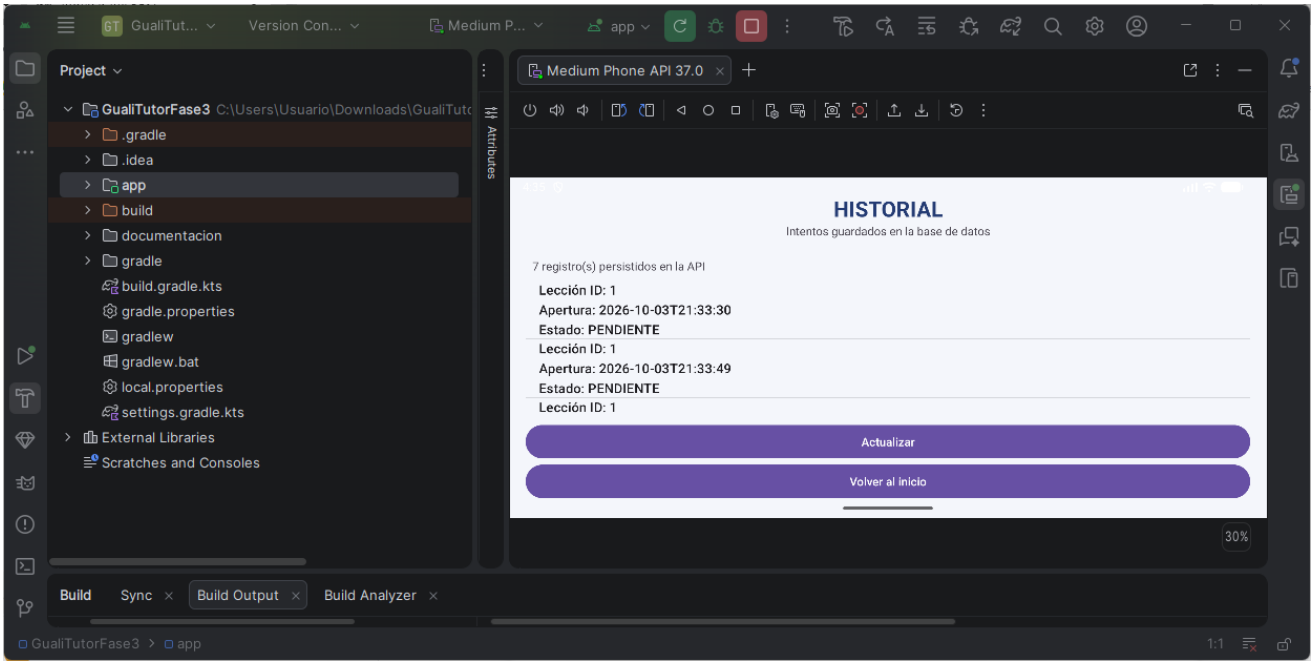


Figura 6. Historial mostrado en el emulador Android.

Paso 7. Consulta de configuración

La sección Configuración muestra los valores recuperados desde la API REST. En la evidencia aparecen dos configuraciones: jdk_home, con la ubicación del JDK, y lecciones_root, con la ruta del directorio de lecciones. Estos valores pertenecen al entorno Windows donde se ejecutó el servidor; pueden variar en otro equipo. El botón Actualizar vuelve a solicitar los datos.

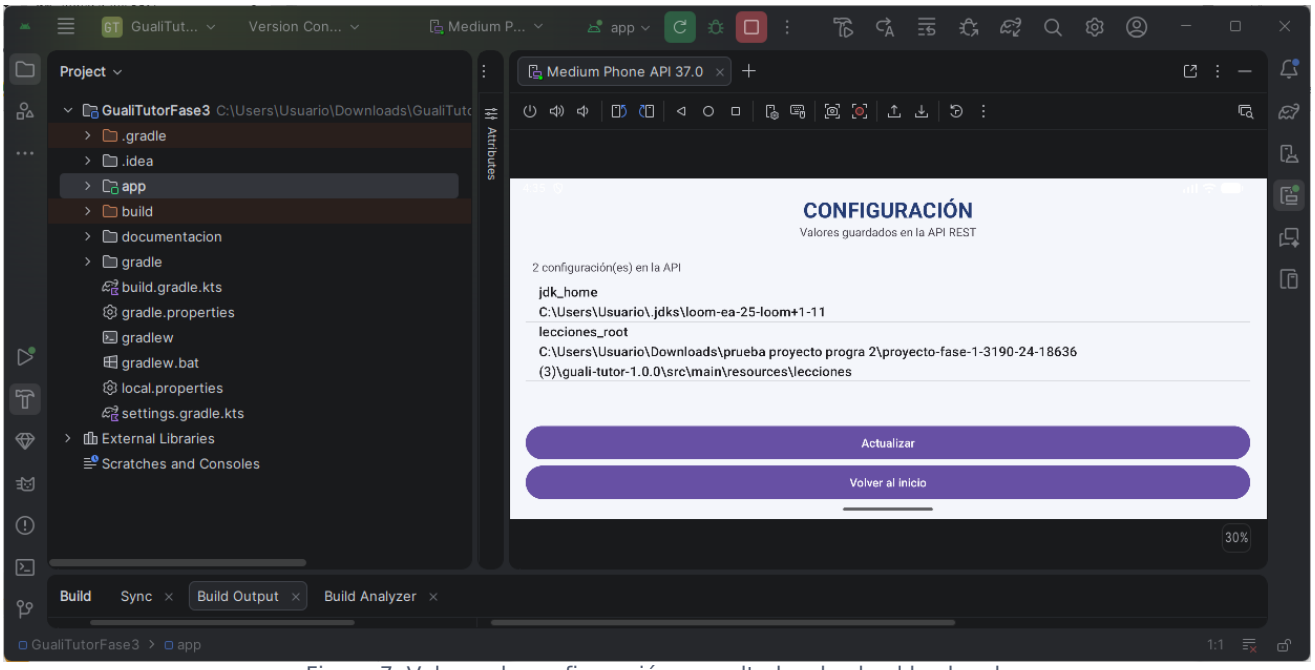


Figura 7. Valores de configuración consultados desde el backend.

Paso 8. Ventana de ayuda

Desde el menú principal se implementó un cuadro de diálogo de ayuda que orienta al usuario: abrir Mis lecciones, seleccionar una lección, consultar el contenido, escribir código y revisar Historial y Configuración. La ventana se cierra mediante Entendido. El texto visible en la captura advierte que la ejecución depende de un servicio compatible que todavía no está disponible en el Front-End; esa limitación se conserva expresamente en la documentación.

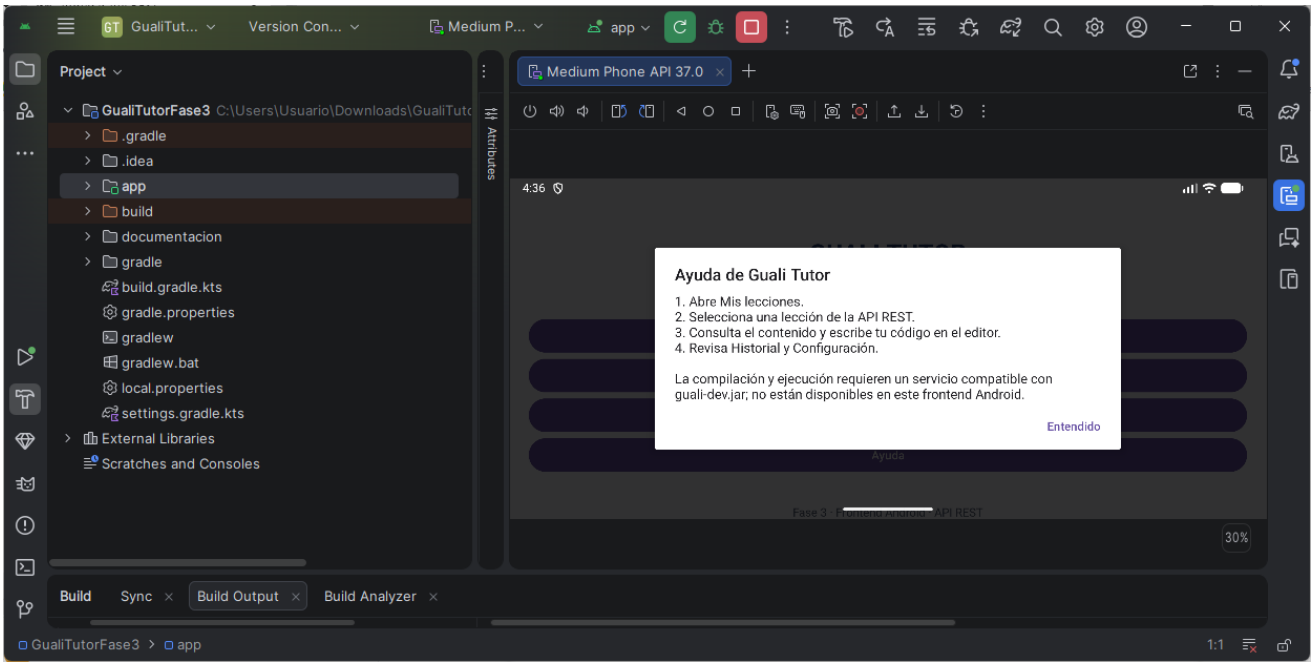


Figura 8. Cuadro de diálogo de ayuda de Guali Tutor.

6. Integración de la aplicación con la API REST

La comunicación entre Android y Spring Boot se realiza por HTTP. El cliente de red ApiClient utiliza como base la dirección del servidor accesible desde el emulador. Las pantallas de lecciones, historial y configuración consultan sus respectivos endpoints GET y procesan la respuesta JSON para actualizar los controles. El editor envía código fuente mediante POST al endpoint de compilación, que devuelve campos de éxito, mensaje y errores. Las peticiones deben ejecutarse fuera del hilo principal de Android y la respuesta se refleja en la interfaz.

Para realizar las pruebas se inició el backend Spring Boot en el puerto 8080 y se verificó el endpoint de estado del compilador. Después se comprobó mediante una solicitud POST que un ejemplo Java podía compilarse y, desde Android, se observó el mensaje de compilación correcta. El servidor de compilación corresponde a un entorno de práctica local; no debe exponerse a Internet para recibir código arbitrario sin aislamiento y límites de recursos.

7. Procedimiento de instalación y prueba

1. Obtener el código del repositorio público y abrir la carpeta del proyecto en Android Studio.

2. Permitir que Gradle sincronice las dependencias y comprobar que el SDK de Android esté instalado.

3. Iniciar la API REST en el equipo de desarrollo, con Java y la conexión a MariaDB configurados. Verificar que el servicio responda en el puerto 8080.

4. Iniciar el emulador Android, seleccionar la configuración app y ejecutar la aplicación.

5. Probar la navegación principal, cargar Mis lecciones, abrir una lección y compilar un ejemplo Java.

6. Consultar Historial y Configuración y verificar que se muestren los registros del servidor.

7. Abrir Ayuda y comprobar que se despliegue el diálogo de instrucciones.

8. Resultados y observaciones

Las evidencias muestran interfaces XML creadas en Android Studio, listado de lecciones, editor de código, consulta de historial y configuración, y ventana de ayuda. En las pruebas del proyecto también se confirmó la compilación de código Java mediante la API. La función Ejecutar sigue pendiente de implementar con aislamiento seguro. Los datos de lecciones e historial son los existentes en la base de datos de prueba; no se debe asumir que son definitivos.

9. Conclusiones

Se desarrolló un Front-End Android organizado por pantallas y conectado a una API REST para consultar información de lecciones, historial y configuración. La separación entre diseños XML, actividades Java y cliente de red facilitó la integración y las pruebas. Las capturas permiten comprobar el aspecto y el funcionamiento observado de los módulos, y delimitan las funciones que aún requieren desarrollo.

10. Referencias y enlaces

Repositorio del proyecto: https://github.com/dsinaym-prog/GualiTutor-Fase3

Tutorial público en Markdown: https://github.com/dsinaym-prog/GualiTutor-Fase3/blob/main/TUTORIAL.md

Documentación oficial de Android Developers: https://developer.android.com/

Documentación oficial de Spring Boot: https://docs.spring.io/spring-boot/